University of Asia Pacific

Department of Computer Science and Engineering

Course Code : CSE 450 (VLSI Lab) Final Project | Spring 2026

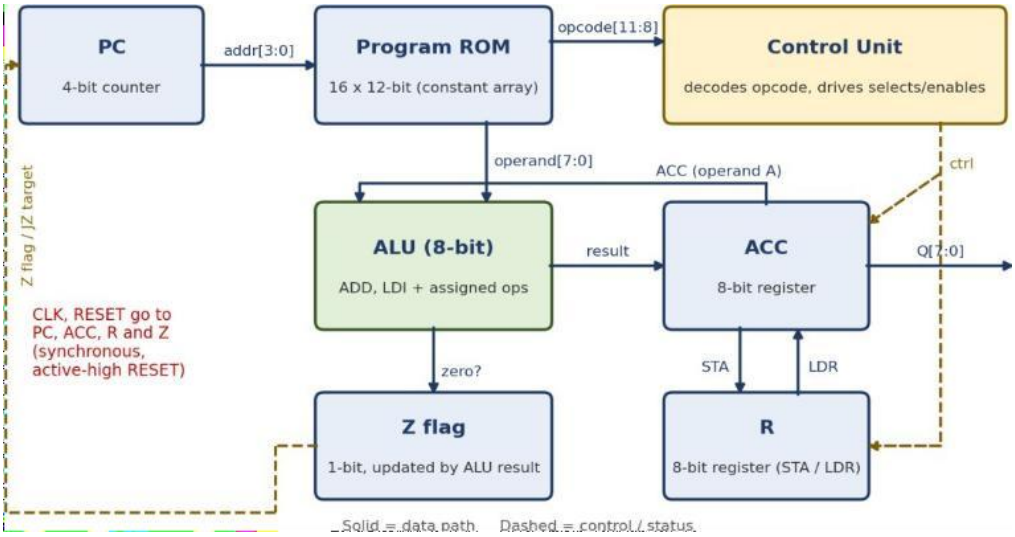
Total Marks	50 (Viva → 30, Report & VrC → 10, Presentation → 10)		
Student Name	MD Rawnak Shalahen Fahim	**ID**	22201063

Final Project: Mini 8-bit Accumulator CPU

Objective. Design and implement a small single-cycle 8-bit accumulator CPU in VHDL. The CPU supports only five instructions: two are mandatory for every student, and the remaining three are assigned to you according to your Student ID (see the "Instruction Set Assignment" page: the last digit of your Student ID decides your set). You must also verify the design with a testbench that runs the program given for your set in Appendix B.

1. System Architecture

The CPU is built from the blocks shown below. Every sequential block (PC, ACC, R, Z) is clocked on the rising edge of CLK and cleared by a synchronous, active-high RESET.



Block	Description
PC	4-bit program counter. Holds the address of the instruction being executed. After every instruction it becomes PC + 1, except when JZ is taken.
Program ROM	16 locations x 12 bits, written as a constant array in VHDL. It is read combinationally using PC (no clock needed). Unused locations contain 0x000 (NOP).
Control Unit	Decodes opcode[11:8] and generates the ALU function select and the load/enable signals for ACC, R, Z and PC.
ALU	8-bit combinational unit working on ACC and the 8-bit operand. Only the operations of your own instruction set are required.
ACC, R, Z	ACC: 8-bit accumulator, also the CPU output Q[7:0]. R: 8-bit holding register (only if your set has STA / LDR). Z: 1-bit flag, 1 when the value written to ACC is 0x00.

2. Instruction Format

Each instruction is 12 bits: **instr[11:8] = opcode** (4 bits) and **instr[7:0] = operand** (8-bit immediate). Every instruction takes exactly one clock cycle (fetch, decode and execute happen in the same cycle, the result is written at the rising edge).

Mandatory instructions (all students):

Mnemonic	Opcode	Operation	Flags
LDI imm	0001	ACC ← imm	Z updated
ADD imm	0010	ACC ← ACC + imm (8-bit, carry discarded)	Z updated

Instruction pool (you implement only the three assigned to you):

Mnemonic	Opcode	Operation	Flags
SUB imm	0011	ACC ← ACC − imm (modulo 256)	Z updated
AND imm	0100	ACC ← ACC AND imm	Z updated
OR imm	0101	ACC ← ACC OR imm	Z updated
XOR imm	0110	ACC ← ACC XOR imm	Z updated
NOT	0111	ACC ← NOT ACC (operand ignored)	Z updated
SHL	1000	ACC ← ACC shifted left by 1, 0 enters LSB (operand ignored)	Z updated
SHR	1001	ACC ← ACC shifted right by 1, 0 enters MSB (operand ignored)	Z updated
STA	1010	R ← ACC (operand ignored, ACC unchanged)	Z unchanged
LDR	1011	ACC ← R (operand ignored)	Z updated
JZ addr	1100	If Z = 1 then PC ← operand[3:0], else PC ← PC + 1	Z unchanged

Note: if your set contains LDR it also contains STA. Any opcode not in your set (including 0000) behaves as NOP: PC ← PC + 1 and nothing else changes.

3. Reset Behaviour

RESET = 1 at a rising edge: PC ← 0, ACC ← 0x00, R ← 0x00, Z ← 1 (RESET has priority over everything). **RESET = 0 at a rising edge:** execute the instruction at ROM[PC]. Between clock edges all registers keep their values.

4. Implementation Requirements

- Use VHDL with IEEE.STD_LOGIC_1164.ALL (IEEE.NUMERIC_STD is allowed for arithmetic).
- Modular design: separate entities for PC, Program ROM, Control Unit, ALU and ACC/R/Z, connected in a top-level CPU entity.
- RESET must be synchronous and active-high. All state changes must occur on the rising edge of CLK.
- The top level must expose CLK, RESET, Q[7:0] (the ACC), the Z flag and, for checking, PC[3:0].
- Implement only the five instructions of your own set. The ROM must hold exactly the Appendix B program of your set; the testbench must generate a clock, apply RESET for one rising edge, and run until the last instruction has executed.

Instruction Set Assignment

How to find your set: take the **last digit of your Student ID** (ID mod 10). That digit is your set number in the table below. Example: ID 22201014 → last digit 4 → Set 4.

Every student implements the two mandatory instructions **LDI** and **ADD** plus the three assigned instructions of his/her set. Total = 5 instructions.

Last digit of ID	Set	Mandatory	Assigned	Complete instruction set of the CPU
0	**Set 0**	LDI, ADD	AND, JZ, NOT	LDI, ADD, AND, JZ, NOT
1	**Set 1**	LDI, ADD	SUB, AND, OR	LDI, ADD, SUB, AND, OR
2	**Set 2**	LDI, ADD	XOR, NOT, SHL	LDI, ADD, XOR, NOT, SHL
3	**Set 3**	LDI, ADD	SHR, SUB, XOR	LDI, ADD, SHR, SUB, XOR
4	**Set 4**	LDI, ADD	STA, LDR, AND	LDI, ADD, STA, LDR, AND

2. Instruction Format

Each instruction is 12 bits: **instr[11:8] = opcode** (4 bits) and **instr[7:0] = operand** (8-bit immediate). Every instruction takes exactly one clock cycle (fetch, decode and execute happen in the same cycle, the result is written at the rising edge).

Mandatory instructions (all students):

Mnemonic	Opcode	Operation	Flags
LDI imm	0001	ACC ← imm	Z updated
ADD imm	0010	ACC ← ACC + imm (8-bit, carry discarded)	Z updated

Instruction pool (you implement only the three assigned to you):

Mnemonic	Opcode	Operation	Flags
SUB imm	0011	ACC ← ACC − imm (modulo 256)	Z updated
AND imm	0100	ACC ← ACC AND imm	Z updated
OR imm	0101	ACC ← ACC OR imm	Z updated
XOR imm	0110	ACC ← ACC XOR imm	Z updated
NOT	0111	ACC ← NOT ACC (operand ignored)	Z updated
SHL	1000	ACC ← ACC shifted left by 1, 0 enters LSB (operand ignored)	Z updated
SHR	1001	ACC ← ACC shifted right by 1, 0 enters MSB (operand ignored)	Z updated
STA	1010	R ← ACC (operand ignored, ACC unchanged)	Z unchanged
LDR	1011	ACC ← R (operand ignored)	Z updated
JZ addr	1100	If Z = 1 then PC ← operand[3:0], else PC ← PC + 1	Z unchanged

Note: if your set contains LDR it also contains STA. Any opcode not in your set (including 0000) behaves as NOP: PC ← PC + 1 and nothing else changes.

3. Reset Behaviour

RESET = 1 at a rising edge: PC ← 0, ACC ← 0x00, R ← 0x00, Z ← 1 (RESET has priority over everything). **RESET = 0 at a rising edge:** execute the instruction at ROM[PC]. Between clock edges all registers keep their values.

4. Implementation Requirements

- Use VHDL with IEEE.STD_LOGIC_1164.ALL (IEEE.NUMERIC_STD is allowed for arithmetic).
- Modular design: separate entities for PC, Program ROM, Control Unit, ALU and ACC/R/Z, connected in a top-level CPU entity.
- RESET must be synchronous and active-high. All state changes must occur on the rising edge of CLK.
- The top level must expose CLK, RESET, Q[7:0] (the ACC), the Z flag and, for checking, PC[3:0].
- Implement only the five instructions of your own set. The ROM must hold exactly the Appendix B program of your set; the testbench must generate a clock, apply RESET for one rising edge, and run until the last instruction has executed.

Instruction Set Assignment

How to find your set: take the **last digit of your Student ID** (ID mod 10). That digit is your set number in the table below. Example: ID 22201014 → last digit 4 → Set 4.

Every student implements the two mandatory instructions **LDI** and **ADD** plus the three assigned instructions of his/her set. Total = 5 instructions.

Last digit of ID	Set	Mandatory	Assigned	Complete instruction set of the CPU
0	**Set 0**	LDI, ADD	AND, JZ, NOT	LDI, ADD, AND, JZ, NOT
1	**Set 1**	LDI, ADD	SUB, AND, OR	LDI, ADD, SUB, AND, OR
2	**Set 2**	LDI, ADD	XOR, NOT, SHL	LDI, ADD, XOR, NOT, SHL
3	**Set 3**	LDI, ADD	SHR, SUB, XOR	LDI, ADD, SHR, SUB, XOR
4	**Set 4**	LDI, ADD	STA, LDR, AND	LDI, ADD, STA, LDR, AND
Last digit of ID	**Set**	**Mandatory**	**Assigned**	**Complete instruction set of the CPU**
5	**Set 5**	LDI, ADD	OR, NOT, SHR	LDI, ADD, OR, NOT, SHR
6	**Set 6**	LDI, ADD	SUB, JZ, SHL	LDI, ADD, SUB, JZ, SHL
7	**Set 7**	LDI, ADD	AND, XOR, SHR	LDI, ADD, AND, XOR, SHR
8	**Set 8**	LDI, ADD	STA, LDR, SUB	LDI, ADD, STA, LDR, SUB
9	**Set 9**	LDI, ADD	NOT, SHL, JZ	LDI, ADD, NOT, SHL, JZ

Appendix B: Required Test Program and Expected Results

Store the program of your set (Appendix B tables are ordered by set number 0–9) in the Program ROM. Apply RESET for one rising edge, then run the CPU for one clock cycle per executed instruction. The ACC, Z, R and next-PC columns show the values after the corresponding rising edge. Addresses that are not listed contain 0x000 (NOP). Operands are in hexadecimal. For NOT, SHL, SHR, STA and LDR the operand field is 0x00.

ROM encoding: instr[11:0] = opcode (4 bit) & operand (8 bit). Example: LDI 0x3C = 0001 00111100 = 0x13C.

Set 0: LDI, ADD, AND, JZ, NOT

ROM contents: 0: LDI 0xA5 → 0x1A5 | 1: AND 0x5A → 0x45A | 2: JZ 0x05 → 0xC05 | 3: ADD 0x01 → 0x201 | 4: ADD 0x01 → 0x201 | 5: NOT → 0x700 | 6: AND 0x0F → 0x40F | 7: JZ 0x00 → 0xC00 | 8: ADD 0x01 → 0x201

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0xA5	0xA5	0	10100101	1
2	1	AND 0x5A	0x00	1	00000000	2
3	2	JZ 0x05 (taken)	0x00	1	00000000	5
4	5	NOT	0xFF	0	11111111	6
5	6	AND 0x0F	0x0F	0	00001111	7
6	7	JZ 0x00 (not taken)	0x0F	0	00001111	8

7	8	ADD 0x01	0x10	0	00010000	9

Set 1: LDI, ADD, SUB, AND, OR

ROM contents: 0: LDI 0x3C → 0x13C | 1: ADD 0x0F → 0x20F | 2: SUB 0x0B → 0x30B | 3: AND 0xF0 → 0x4F0 | 4: OR 0x0A → 0x50A | 5: SUB 0x4A → 0x34A | 6: SUB 0x01 → 0x301

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0x3C	0x3C	0	00111100	1
2	1	ADD 0x0F	0x4B	0	01001011	2
3	2	SUB 0x0B	0x40	0	01000000	3
4	3	AND 0xF0	0x40	0	01000000	4
5	4	OR 0x0A	0x4A	0	01001010	5
6	5	SUB 0x4A	0x00	1	00000000	6
7	6	SUB 0x01	0xFF	0	11111111	7

Set 2: LDI, ADD, XOR, NOT, SHL

ROM contents: 0: LDI 0x55 → 0x155 | 1: XOR 0xFF → 0x6FF | 2: NOT → 0x700 | 3: SHL → 0x800 | 4: ADD 0x56 → 0x256 | 5: XOR 0x81 → 0x681 | 6: SHL → 0x800

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0x55	0x55	0	01010101	1
2	1	XOR 0xFF	0xAA	0	10101010	2
3	2	NOT	0x55	0	01010101	3
4	3	SHL	0xAA	0	10101010	4
5	4	ADD 0x56	0x00	1	00000000	5
6	5	XOR 0x81	0x81	0	10000001	6
7	6	SHL	0x02	0	00000010	7

Set 3: LDI, ADD, SHR, SUB, XOR

ROM contents: 0: LDI 0x80 → 0x180 | 1: SHR → 0x900 | 2: SUB 0x10 → 0x310 | 3: ADD 0x0F → 0x20F | 4: XOR 0x3F → 0x63F | 5: ADD 0x07 → 0x207 | 6: SHR → 0x900

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0x80	0x80	0	10000000	1
2	1	SHR	0x40	0	01000000	2

3	2	SUB 0x10	0x30	0	00110000	3
4	3	ADD 0x0F	0x3F	0	00111111	4
5	4	XOR 0x3F	0x00	1	00000000	5
6	5	ADD 0x07	0x07	0	00000111	6
7	6	SHR	0x03	0	00000011	7

Set 4: LDI, ADD, STA, LDR, AND

ROM contents: 0: LDI 0x1E → 0x11E | 1: STA → 0xA00 | 2: ADD 0x22 → 0x222 | 3: AND 0x60 → 0x460 | 4: LDR → 0xB00 | 5: ADD 0x01 → 0x201 | 6: AND 0x0F → 0x40F | 7: STA → 0xA00 | 8: LDI 0x00 → 0x100 | 9: LDR → 0xB00

Cycle	PC	Instruction executed	ACC (hex)	Z	R (hex)	ACC (bin)	Next PC
0	-	RESET = 1	0x00	1	0x00	00000000	0
1	0	LDI 0x1E	0x1E	0	0x00	00011110	1
2	1	STA	0x1E	0	0x1E	00011110	2
3	2	ADD 0x22	0x40	0	0x1E	01000000	3
4	3	AND 0x60	0x40	0	0x1E	01000000	4
5	4	LDR	0x1E	0	0x1E	00011110	5
6	5	ADD 0x01	0x1F	0	0x1E	00011111	6
7	6	AND 0x0F	0x0F	0	0x1E	00001111	7
8	7	STA	0x0F	0	0x0F	00001111	8
9	8	LDI 0x00	0x00	1	0x0F	00000000	9
10	9	LDR	0x0F	0	0x0F	00001111	10

Set 5: LDI, ADD, OR, NOT, SHR

ROM contents: 0: LDI 0x0F → 0x10F | 1: OR 0xA0 → 0x5A0 | 2: NOT → 0x700 | 3: SHR → 0x900 | 4: ADD 0xD8 → 0x2D8 | 5: OR 0x81 → 0x581 | 6: SHR → 0x900

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0x0F	0x0F	0	00001111	1
2	1	OR 0xA0	0xAF	0	10101111	2
3	2	NOT	0x50	0	01010000	3
4	3	SHR	0x28	0	00101000	4
5	4	ADD 0xD8	0x00	1	00000000	5

6	5	OR 0x81	0x81	0	10000001	6
7	6	SHR	0x40	0	01000000	7

Set 6: LDI, ADD, SUB, JZ, SHL

ROM contents: 0: LDI 0x02 → 0x102 | 1: SUB 0x01 → 0x301 | 2: JZ 0x07 → 0xC07 | 3: SUB 0x01 → 0x301 | 4: JZ 0x07 → 0xC07 | 5: ADD 0xAA → 0x2AA | 6: ADD 0xAA → 0x2AA | 7: LDI 0x21 → 0x121 | 8: SHL → 0x800 | 9: ADD 0x01 → 0x201

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0x02	0x02	0	00000010	1
2	1	SUB 0x01	0x01	0	00000001	2
3	2	JZ 0x07 (not taken)	0x01	0	00000001	3
4	3	SUB 0x01	0x00	1	00000000	4
5	4	JZ 0x07 (taken)	0x00	1	00000000	7
6	7	LDI 0x21	0x21	0	00100001	8
7	8	SHL	0x42	0	01000010	9
8	9	ADD 0x01	0x43	0	01000011	10

Set 7: LDI, ADD, AND, XOR, SHR

ROM contents: 0: LDI 0xF3 → 0x1F3 | 1: AND 0x3C → 0x43C | 2: XOR 0x11 → 0x611 | 3: SHR → 0x900 | 4: ADD 0x30 → 0x230 | 5: XOR 0x40 → 0x640

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0xF3	0xF3	0	11110011	1
2	1	AND 0x3C	0x30	0	00110000	2
3	2	XOR 0x11	0x21	0	00100001	3
4	3	SHR	0x10	0	00010000	4
5	4	ADD 0x30	0x40	0	01000000	5
6	5	XOR 0x40	0x00	1	00000000	6

Set 8: LDI, ADD, STA, LDR, SUB

ROM contents: 0: LDI 0x64 → 0x164 | 1: STA → 0xA00 | 2: SUB 0x14 → 0x314 | 3: ADD 0x0A → 0x20A | 4: STA → 0xA00 | 5: SUB 0x5A → 0x35A | 6: LDR → 0xB00 | 7: SUB 0x60 → 0x360

Cycle	PC	Instruction executed	ACC (hex)	Z	R (hex)	ACC (bin)	Next PC

0	-	RESET = 1	0x00	1	0x00	00000000	0
1	0	LDI 0x64	0x64	0	0x00	01100100	1
2	1	STA	0x64	0	0x64	01100100	2
3	2	SUB 0x14	0x50	0	0x64	01010000	3
4	3	ADD 0x0A	0x5A	0	0x64	01011010	4
5	4	STA	0x5A	0	0x5A	01011010	5
6	5	SUB 0x5A	0x00	1	0x5A	00000000	6
7	6	LDR	0x5A	0	0x5A	01011010	7
8	7	SUB 0x60	0xFA	0	0x5A	11111010	8

Set 9: LDI, ADD, NOT, SHL, JZ

ROM contents: 0: LDI 0xFF → 0x1FF | 1: NOT → 0x700 | 2: JZ 0x05 → 0xC05 | 3: ADD 0x77 → 0x277 | 4: ADD 0x77 → 0x277 | 5: ADD 0x11 → 0x211 | 6: SHL → 0x800 | 7: JZ 0x00 → 0xC00 | 8: SHL → 0x800

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0xFF	0xFF	0	11111111	1
2	1	NOT	0x00	1	00000000	2
3	2	JZ 0x05 (taken)	0x00	1	00000000	5
4	5	ADD 0x11	0x11	0	00010001	6
5	6	SHL	0x22	0	00100010	7
6	7	JZ 0x00 (not taken)	0x22	0	00100010	8
7	8	SHL	0x44	0	01000100	9

Submission Guidelines

Submit your VHDL source files, testbench and a project report within the 2-week period. Include this project description (question, assignment page and Appendix B) at the front of your report and start your own report on a new page after it. The report must include, but is not limited to, the following points.

- Your Student ID, assigned set number and the three assigned instructions (highlighted in yellow color)
- Attach your entity code (all modules), testbench code and output visuals (waveform with PC, ACC, Z and R if used)
- Challenges faced during the experiment, and conclusion

Project Report

Mini 8-bit Accumulator CPU in VHDL

CSE 450 (VLSI Lab) | Final Project | Spring 2026

---

1. Student Information and Assigned Instruction Set

This project implements a small single-cycle 8-bit accumulator Central Processing Unit (CPU) using VHDL. The last digit of my Student ID is 3, therefore my assigned instruction set is Set 3.

Item	Details
Student Name	MD Rawnak Shalahen Fahim
Student ID	22201063
Assigned Set	**Set 3**
Mandatory instructions	LDI, ADD
Assigned instructions	**SHR, SUB, XOR**
Complete instruction set	**LDI, ADD, SHR, SUB, XOR**

Each instruction is 12 bits wide. Bits [11:8] contain the 4-bit opcode, and bits [7:0] contain the 8-bit operand. Each instruction is executed in one clock cycle.

Mnemonic	Opcode	Operation	Zero flag (Z)
LDI imm	0001	ACC ← imm	Z Updated
ADD imm	0010	ACC ← ACC + imm (8-bit; carry discarded)	Z Updated
SUB imm	0011	ACC ← ACC − imm modulo 256	Z Updated
SHR	1001	ACC ← ACC shifted right by one; zero enters the most significant bit	Z Updated
XOR imm	0110	ACC ← ACC XOR imm (bitwise exclusive OR)	Z Updated

Table 1: *Instruction set implemented by this CPU (Set 3)*

2. Design Overview

The CPU is organized into reusable VHDL modules. The program counter addresses the program ROM; the instruction decoder splits each fetched instruction; the control unit selects the operation; the ALU computes results; and the accumulator and zero-flag modules provide the outputs.

The design uses a synchronous, active-high reset. At a rising edge where RESET = 1, PC and ACC are cleared and Z is set to 1. When RESET = 0, the instruction at ROM[PC] is executed. The Set 3 instruction set does not use the R holding register or conditional jumps.

2.1 Module hierarchy

Module	Purpose
program_counter	Uses a full-adder incrementer and register to advance the program address.
program_rom	Stores the Set 3 instruction sequence at addresses 0–6; other addresses return 0x000.
instruction_decoder	Splits the 12-bit instruction into opcode and operand.
control_unit	Generates LOAD_EN, ALU_OP and ACC_WR control signals.
alu_8bit	Performs ADD, XOR, SUB and SHR using structural components.
accumulator / register_8bit	Stores the 8-bit accumulator value through the register and logic components.
zero_flag	Sets ZERO high when all accumulator bits are zero.
cpu_top	Connects the CPU modules and exposes Q, Z and PC.

cpu_top_tb	Applies reset, runs the Set 3 program and checks expected outputs.

Table 2: *Module hierarchy (lowest level first)*

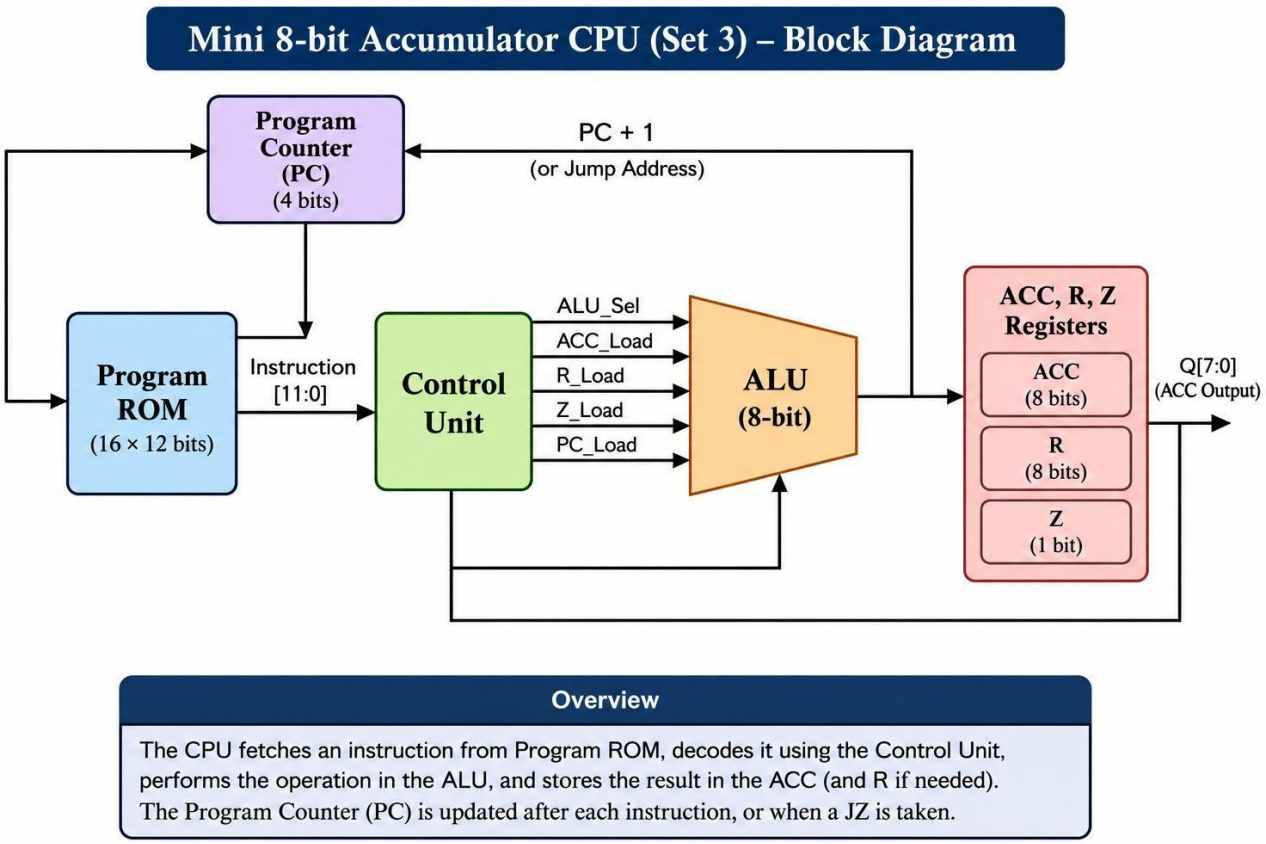


Figure 1: *Block diagram of the Mini 8-bit Accumulator CPU.*

2.2 Instruction encoding

The 12-bit instruction is formed by concatenating the 4-bit opcode and the 8-bit operand. For example, LDI 0x80 is encoded as opcode 0001 followed by operand 10000000, producing 0x180. For SHR, the operand is ignored and is set to 0x00.

Address	Instruction	Opcode + operand	12-bit word
0	LDI 0x80	0001 & 10000000	0x180
1	SHR	1001 & 00000000	0x900
2	SUB 0x10	0011 & 00010000	0x310
3	ADD 0x0F	0010 & 00001111	0x20F
4	XOR 0x3F	0110 & 00111111	0x63F
5	ADD 0x07	0010 & 00000111	0x207
6	SHR	1001 & 00000000	0x900
7–15	NOP / unused	0000 & 00000000	0x000

2.3 Program ROM

The Program ROM contains exactly the Set 3 program specified in Appendix B. It is read combinationally using PC. Unused ROM locations return 0x000, which is treated as a no-operation (NOP) instruction.

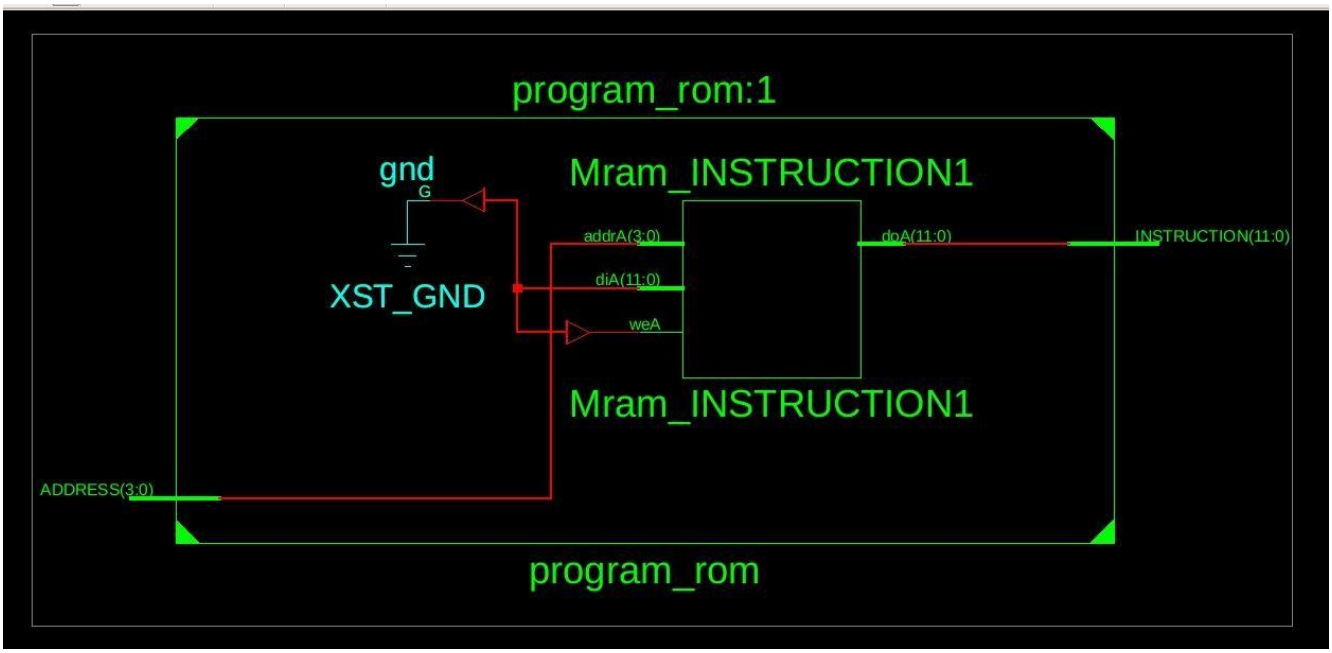


Figure 2: *Program ROM*

3. Implementation

This section uses the VHDL source files from the uploaded cpu_design project. The design is built from structural modules and basic logic components. The listings below are copied from the supplied project so the report matches the code you intend to submit.

3.1 Program Counter (program_counter.vhd)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity program_counter is
    Port (
        CLK   : in  STD_LOGIC;
        RESET : in  STD_LOGIC;
        --LOAD  : in  STD_LOGIC;
        --D     : in  STD_LOGIC_VECTOR(3 downto 0);
        PC    : out STD_LOGIC_VECTOR(3 downto 0)
    );
end program_counter;
architecture Structural of program_counter is

    component full_adder_8bit
        Port (
            A    : in  STD_LOGIC_VECTOR(7 downto 0);
            B    : in  STD_LOGIC_VECTOR(7 downto 0);
            CIN  : in  STD_LOGIC;
            SUM  : out STD_LOGIC_VECTOR(7 downto 0);
            COUT : out STD_LOGIC
        );
    end component;
    component register_8bit
        Port (
            D     : in  STD_LOGIC_VECTOR(7 downto 0);
            CLK   : in  STD_LOGIC;
            RESET : in  STD_LOGIC;
            Q     : out STD_LOGIC_VECTOR(7 downto 0)
        );
    end component;

    signal PC_CURRENT : STD_LOGIC_VECTOR(7 downto 0);
    signal PC_NEXT   : STD_LOGIC_VECTOR(7 downto 0);
    signal CARRY_OUT  : STD_LOGIC;
begin
```

```
    INCREMENTER: full_adder_8bit
        port map (
            A    => PC_CURRENT,
            B    => "00000001",
            CIN  => '0',
            SUM  => PC_NEXT,
            COUT => CARRY_OUT
        );
    PC_REGISTER: register_8bit
        port map (
            D     => PC_NEXT,
            CLK   => CLK,
            RESET => RESET,
            Q     => PC_CURRENT
        );

    PC <= PC_CURRENT(3 downto 0);

end Structural;
```

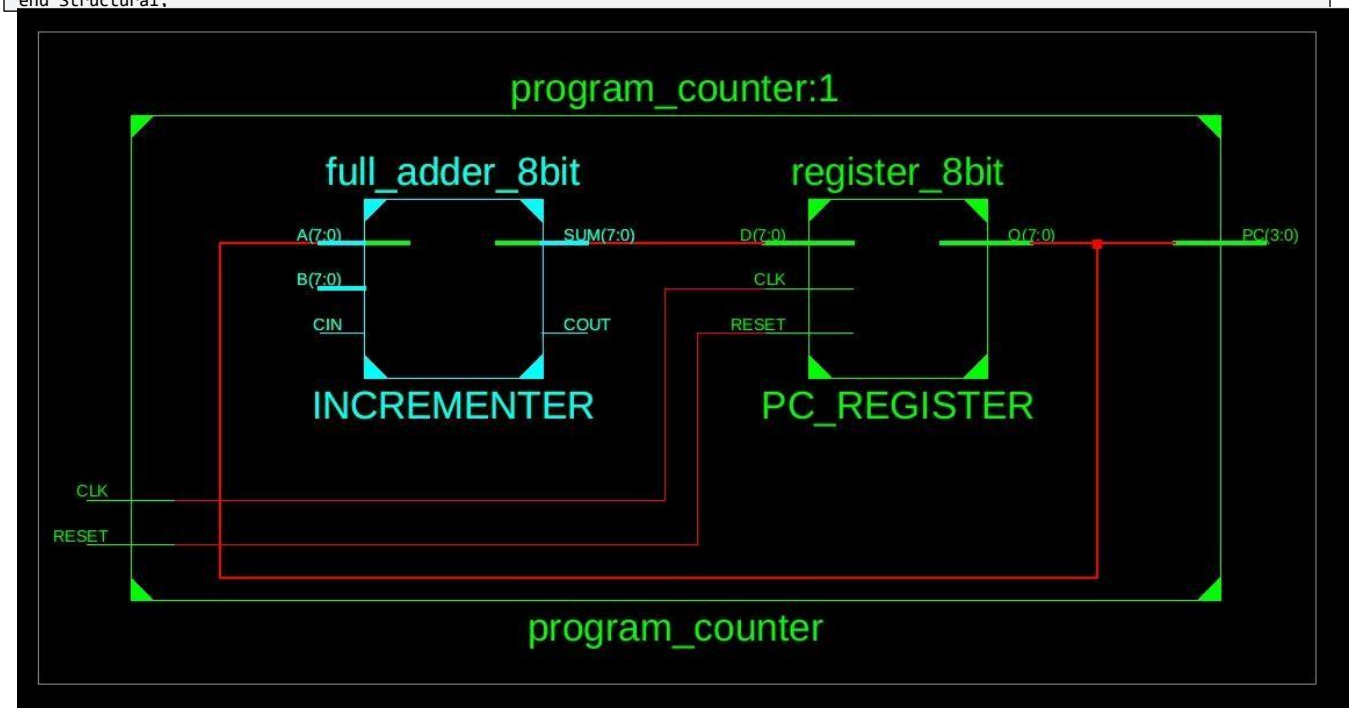


Figure 3: *Program Counter*

The program counter uses an 8-bit register and full-adder incrementer internally, while exposing the lower four bits as the ROM address. The program has no jump instruction, so the address increments sequentially.

3.2 Program ROM (program_rom.vhd)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity program_rom is
    Port (
        ADDRESS     : in  STD_LOGIC_VECTOR(3 downto 0);
        INSTRUCTION : out STD_LOGIC_VECTOR(11 downto 0)
    );
end program_rom;
architecture Behavioral of program_rom is
begin
    process(ADDRESS)
    begin
        case ADDRESS is
            when "0000" => INSTRUCTION <= x"180"; -- LDI 0x80
            when "0001" => INSTRUCTION <= x"900"; -- SHR
            when "0010" => INSTRUCTION <= x"310"; -- SUB 0x10
            when "0011" => INSTRUCTION <= x"20F"; -- ADD 0x0F
when "0100" => INSTRUCTION <= x"63F"; -- XOR 0x3F
            when "0101" => INSTRUCTION <= x"207"; -- ADD 0x07
            when "0110" => INSTRUCTION <= x"900"; -- SHR
            when others => INSTRUCTION <= x"000"; -- NOP
```

```
        end case;
    end process;
end Behavioral;
```

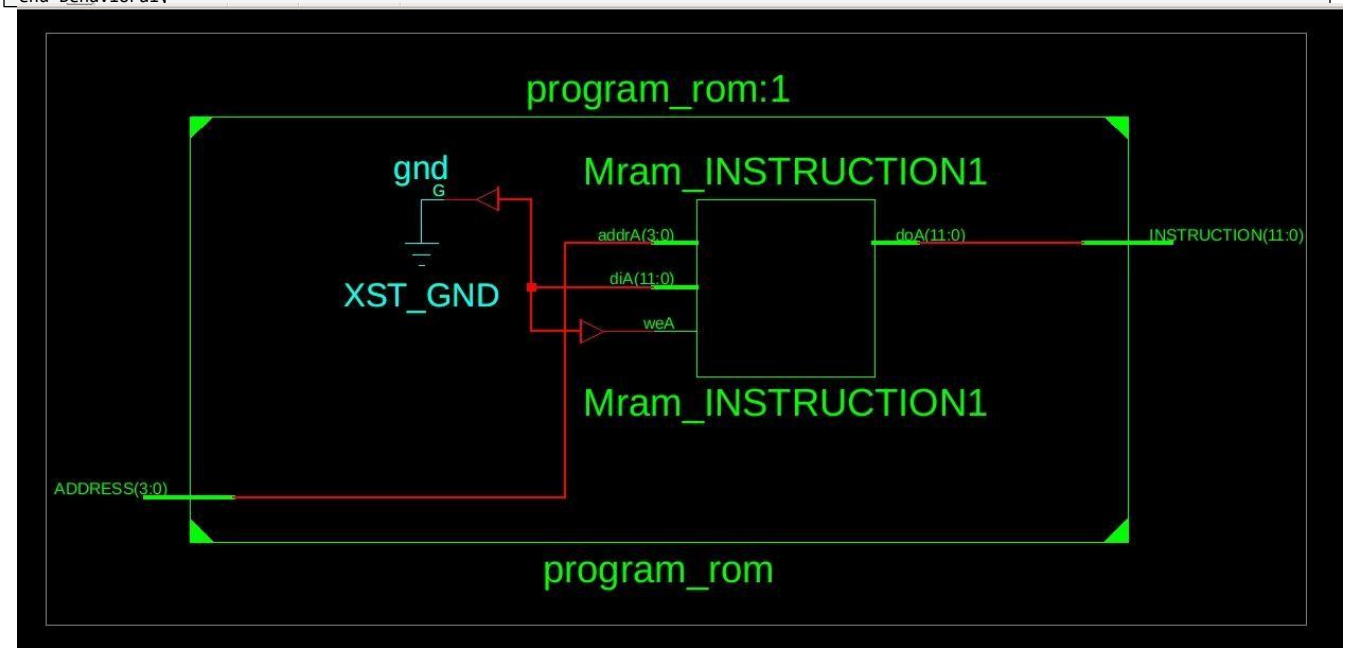


Figure 4: *Program ROM*

3.3 Instruction Decoder and Control Unit

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity instruction_decoder is
    Port (
        INSTRUCTION : in  STD_LOGIC_VECTOR(11 downto 0);
        OPCODE      : out STD_LOGIC_VECTOR(3 downto 0);
        OPERAND     : out STD_LOGIC_VECTOR(7 downto 0)
    );
end instruction_decoder;
architecture Behavioral of instruction_decoder is
begin
    OPCODE  <= INSTRUCTION(11 downto 8);
    OPERAND <= INSTRUCTION(7 downto 0);
end Behavioral;
```

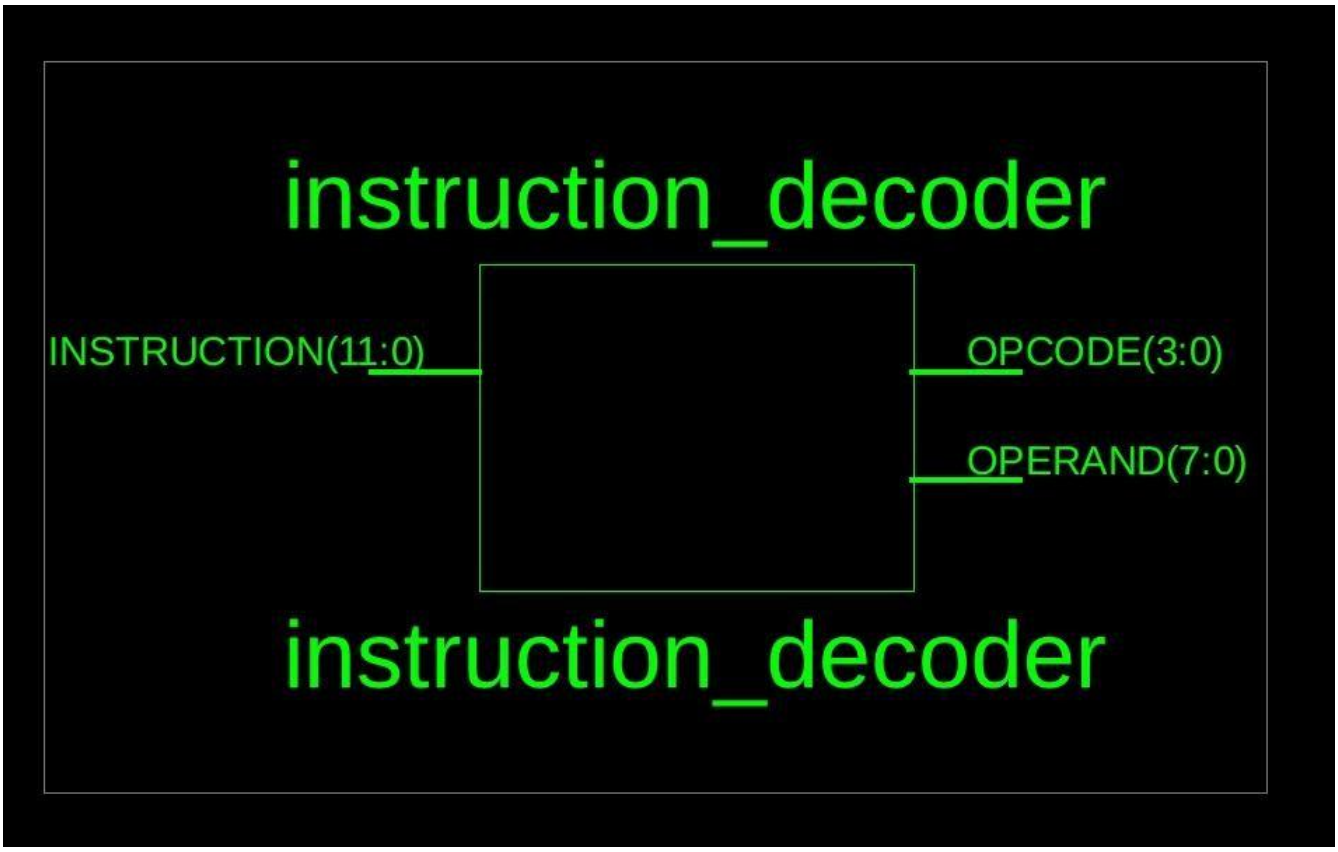


Figure 5: *Instruction Decoder*

The instruction decoder separates the 12-bit instruction into a 4-bit opcode and an 8-bit operand. The control unit maps LDI to a direct load and maps ADD, XOR, SUB and SHR to the ALU operation selector. Unsupported opcodes are treated as NOP.

3.4 ALU (alu_8bit.vhd)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity alu_8bit is
    Port (
        A      : in  STD_LOGIC_VECTOR(7 downto 0);   -- ACC
        B      : in  STD_LOGIC_VECTOR(7 downto 0);   -- operand
        OP     : in  STD_LOGIC_VECTOR(1 downto 0);   -- 00 ADD, 01 XOR, 10 SUB, 11 SHR
        RESULT : out STD_LOGIC_VECTOR(7 downto 0)
    );
end alu_8bit;
architecture Structural of alu_8bit is
    component full_adder_8bit
        Port (
            A    : in  STD_LOGIC_VECTOR(7 downto 0);
            B    : in  STD_LOGIC_VECTOR(7 downto 0);
            CIN  : in  STD_LOGIC;
            SUM  : out STD_LOGIC_VECTOR(7 downto 0);
            COUT : out STD_LOGIC
        );
    end component;
    component xor_gate
        Port ( A : in STD_LOGIC; B : in STD_LOGIC; Y : out STD_LOGIC );
    end component;
    component not_gate
        Port ( A : in STD_LOGIC; Y : out STD_LOGIC );
    end component;
    signal ADD_R, XOR_R, SUB_R, SHR_R : STD_LOGIC_VECTOR(7 downto 0);
    signal B_NOT : STD_LOGIC_VECTOR(7 downto 0);   -- ~B, needed for SUB
    signal CARRY_UNUSED  : STD_LOGIC;
    signal CARRY_UNUSED2 : STD_LOGIC;
begin
    -- ADD : A + B (carry discarded)
    ADDER: full_adder_8bit
```

```
        port map (A => A, B => B, CIN => '0', SUM => ADD_R, COUT => CARRY_UNUSED);
    -- XOR bit by bit, and invert every bit of B (for SUB)
    GEN_LOGIC: for i in 0 to 7 generate
        X: xor_gate port map (A => A(i), B => B(i), Y => XOR_R(i));
        N: not_gate port map (A => B(i), Y => B_NOT(i));
    end generate;
    -- SUB : A - B = A + (~B) + 1   (two's complement, modulo 256)
    SUBTRACTOR: full_adder_8bit
        port map (A => A, B => B_NOT, CIN => '1', SUM => SUB_R, COUT => CARRY_UNUSED2);

    -- SHR : logical shift right by 1, 0 enters the MSB
    SHR_R <= '0' & A(7 downto 1);

    with OP select
        RESULT <= ADD_R when "00",
                  XOR_R when "01",
                  SUB_R when "10",
                  SHR_R when others;
end Structural;
```

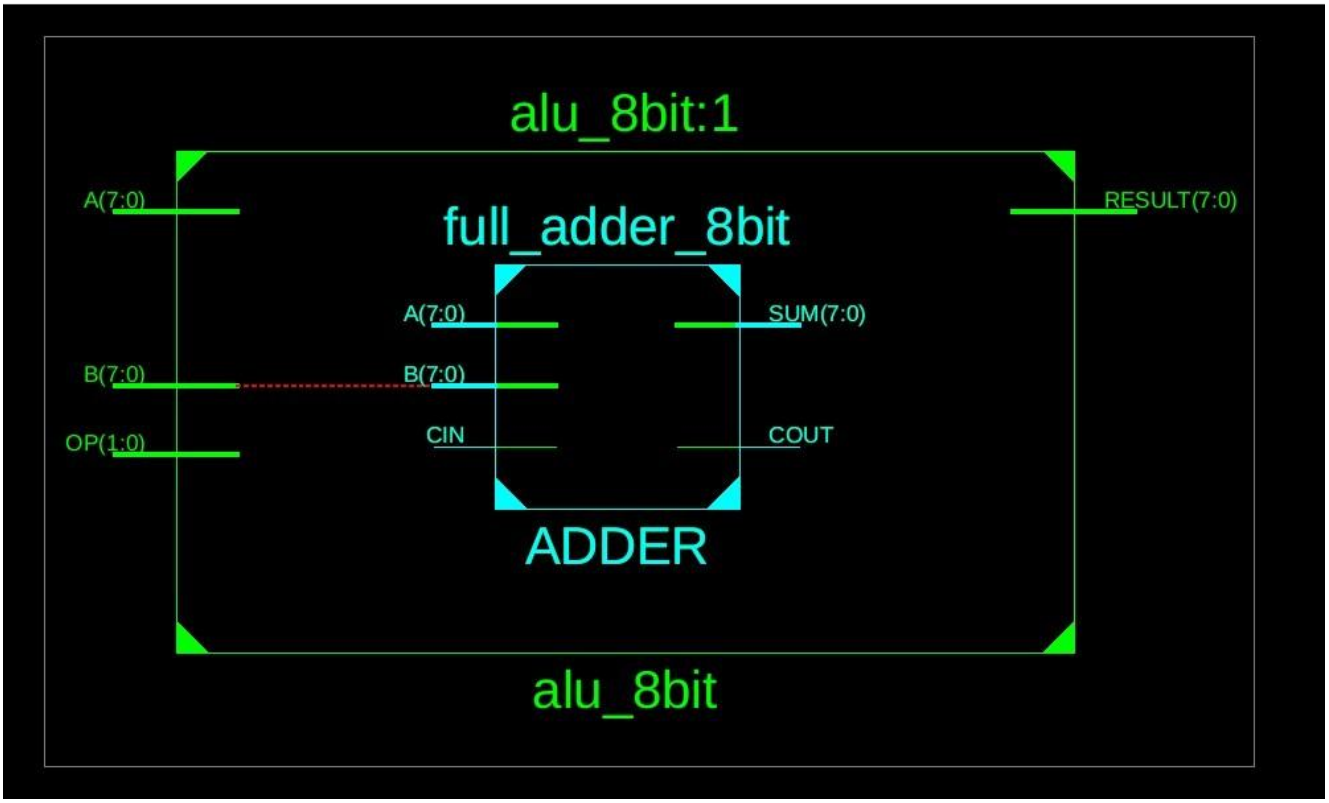


Figure 6: *ALU 8-bit*

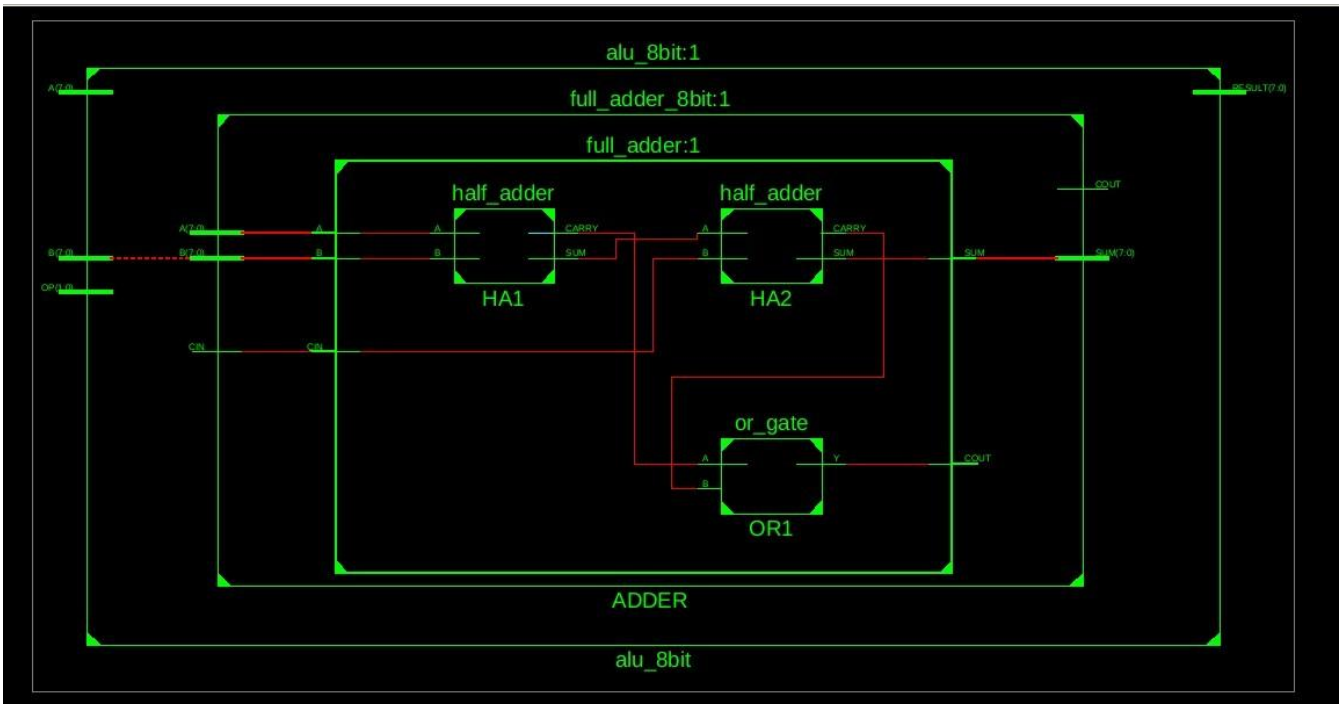


Figure 7: *ALU 8-bit Full Structure*

The ALU in the supplied project is structural: it uses an 8-bit full adder for addition and subtraction, XOR and NOT gates for bitwise operations, and wiring for logical shift right. The 8-bit result discards carry-out.

3.5 Accumulator, Register, Zero Flag and Top-Level CPU

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity alu_8bit is
    Port (
        A      : in  STD_LOGIC_VECTOR(7 downto 0);   -- ACC
        B      : in  STD_LOGIC_VECTOR(7 downto 0);   -- operand
        OP     : in  STD_LOGIC_VECTOR(1 downto 0);   -- 00 ADD, 01 XOR, 10 SUB, 11 SHR
        RESULT : out STD_LOGIC_VECTOR(7 downto 0)
    );
end alu_8bit;
architecture Structural of alu_8bit is
    component full_adder_8bit
        Port (
            A    : in  STD_LOGIC_VECTOR(7 downto 0);
            B    : in  STD_LOGIC_VECTOR(7 downto 0);
            CIN  : in  STD_LOGIC;
            SUM  : out STD_LOGIC_VECTOR(7 downto 0);
            COUT : out STD_LOGIC
        );
    end component;
    component xor_gate
        Port ( A : in STD_LOGIC; B : in STD_LOGIC; Y : out STD_LOGIC );
    end component;
    component not_gate
        Port ( A : in STD_LOGIC; Y : out STD_LOGIC );
    end component;
    signal ADD_R, XOR_R, SUB_R, SHR_R : STD_LOGIC_VECTOR(7 downto 0);
    signal B_NOT : STD_LOGIC_VECTOR(7 downto 0);   -- ~B, needed for SUB
    signal CARRY_UNUSED  : STD_LOGIC;
    signal CARRY_UNUSED2 : STD_LOGIC;
begin
    -- ADD : A + B (carry discarded)
    ADDER: full_adder_8bit
        port map (A => A, B => B, CIN => '0', SUM => ADD_R, COUT => CARRY_UNUSED);
    -- XOR bit by bit, and invert every bit of B (for SUB)
    GEN_LOGIC: for i in 0 to 7 generate
        X: xor_gate port map (A => A(i), B => B(i), Y => XOR_R(i));
        N: not_gate port map (A => B(i), Y => B_NOT(i));
    end generate;
    -- SUB : A - B = A + (~B) + 1   (two's complement, modulo 256)
```

```
    SUBTRACTOR: full_adder_8bit
        port map (A => A, B => B_NOT, CIN => '1', SUM => SUB_R, COUT => CARRY_UNUSED2);

    -- SHR : logical shift right by 1, 0 enters the MSB
    SHR_R <= '0' & A(7 downto 1);

    with OP select
        RESULT <= ADD_R when "00",
                  XOR_R when "01",
                  SUB_R when "10",
                  SHR_R when others;
end Structural;
```

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity cpu_top is
    Port (
        CLK   : in  STD_LOGIC;
        RESET : in  STD_LOGIC;
        Q     : out STD_LOGIC_VECTOR(7 downto 0);
        Z     : out STD_LOGIC;
        PC    : out STD_LOGIC_VECTOR(3 downto 0)
    );
end cpu_top;
architecture Structural of cpu_top is

    signal PC_VALUE    : STD_LOGIC_VECTOR(3 downto 0);
    signal INSTRUCTION : STD_LOGIC_VECTOR(11 downto 0);
    signal OPCODE      : STD_LOGIC_VECTOR(3 downto 0);
    signal OPERAND     : STD_LOGIC_VECTOR(7 downto 0);
    signal LOAD_EN     : STD_LOGIC;
    signal ALU_OP      : STD_LOGIC_VECTOR(1 downto 0);
    signal ACC_WR      : STD_LOGIC;
    signal ALU_RESULT  : STD_LOGIC_VECTOR(7 downto 0);
    signal ACC_NEXT   : STD_LOGIC_VECTOR(7 downto 0);
    signal ACC_VALUE   : STD_LOGIC_VECTOR(7 downto 0);
    signal Z_VALUE    : STD_LOGIC;
begin
    ROM_UNIT : entity work.program_rom
        port map (ADDRESS => PC_VALUE, INSTRUCTION => INSTRUCTION);

    DECODER_UNIT : entity work.instruction_decoder
        port map (INSTRUCTION => INSTRUCTION, OPCODE => OPCODE, OPERAND => OPERAND);

    CONTROL_UNIT : entity work.control_unit
        port map (OPCODE => OPCODE, LOAD_EN => LOAD_EN, ALU_OP => ALU_OP, ACC_WR => ACC_WR);

    ALU_UNIT : entity work.alu_8bit
        port map (A => ACC_VALUE, B => OPERAND, OP => ALU_OP, RESULT => ALU_RESULT);
    -- LDI -> operand, ADD/SUB/XOR/SHR -> ALU result, NOP -> hold
    ACC_NEXT <= OPERAND    when LOAD_EN = '1' else
                ALU_RESULT when ACC_WR  = '1' else
                ACC_VALUE;

    ACC_UNIT : entity work.accumulator
        port map (D => ACC_NEXT, CLK => CLK, RESET => RESET, Q => ACC_VALUE);
    ZERO_UNIT : entity work.zero_flag
        port map (A => ACC_VALUE, ZERO => Z_VALUE);

    PC_UNIT : entity work.program_counter
        port map (CLK => CLK, RESET => RESET, PC => PC_VALUE);

    Q  <= ACC_VALUE;
    Z  <= Z_VALUE;
    PC <= PC_VALUE;
end Structural;
```

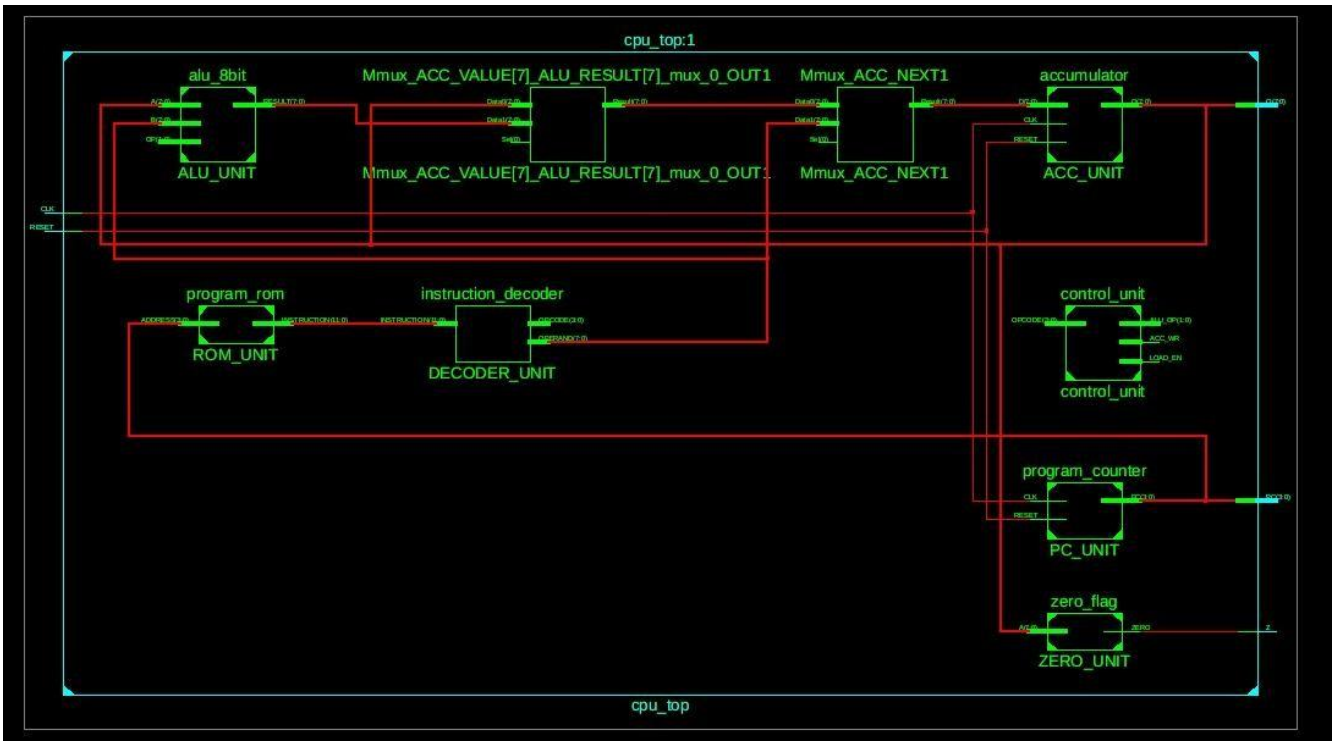


Figure : *CPU Top*

Source file: accumulator.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity accumulator is
    Port (
        D     : in  STD_LOGIC_VECTOR(7 downto 0);
        CLK   : in  STD_LOGIC;
        RESET : in  STD_LOGIC;
        Q     : out STD_LOGIC_VECTOR(7 downto 0)
    );
end accumulator;
architecture Structural of accumulator is
begin

    ACC_REGISTER : entity work.register_8bit
        port map (
            D     => D,
            CLK   => CLK,
            RESET => RESET,
            Q     => Q
        );

end Structural;
```

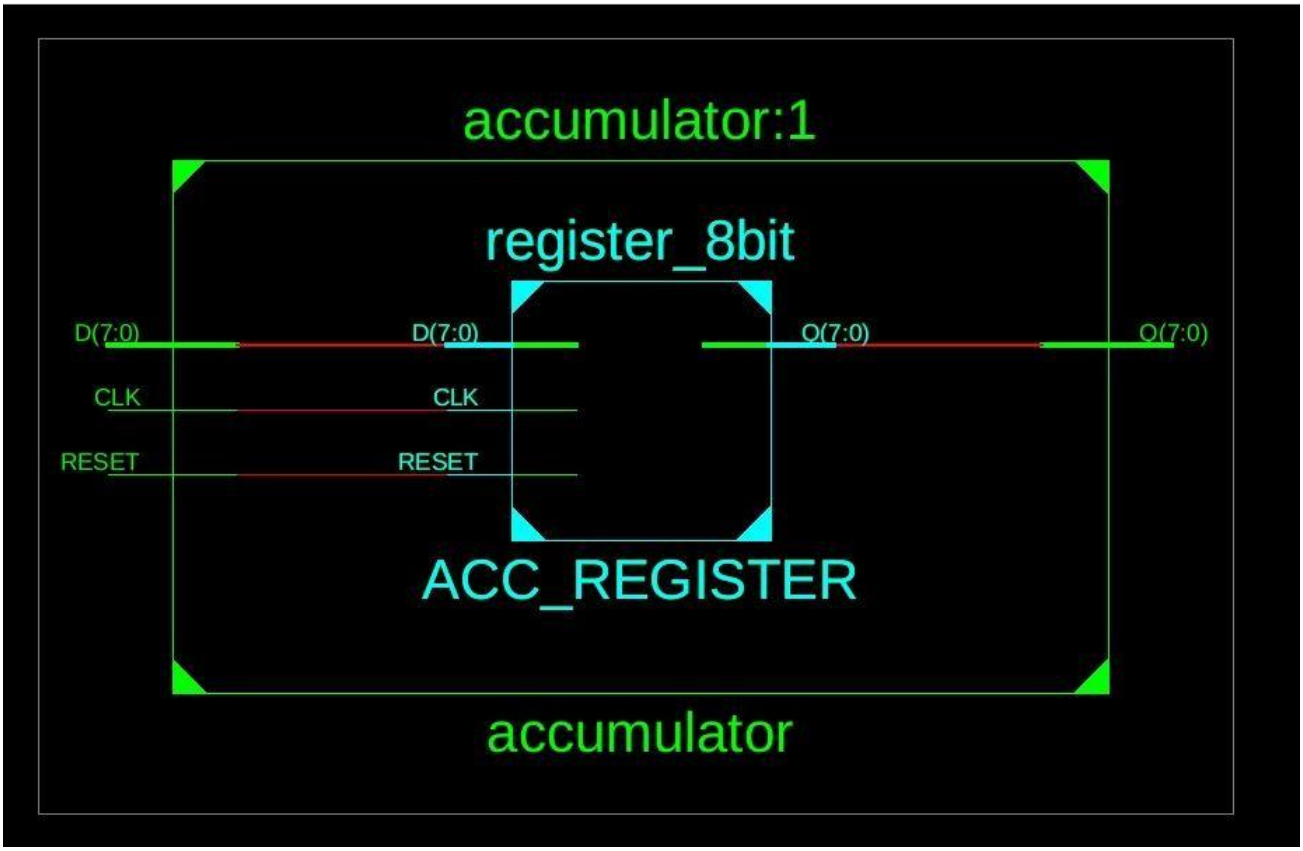


Figure 9: *Accumulator*

Source file: register_8bit.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity register_8bit is
    Port (
        D     : in  STD_LOGIC_VECTOR(7 downto 0);
        CLK   : in  STD_LOGIC;
        RESET : in  STD_LOGIC;
        Q     : out STD_LOGIC_VECTOR(7 downto 0)
    );
end register_8bit;
architecture Structural of register_8bit is

    signal REG_DATA : STD_LOGIC_VECTOR(7 downto 0);
    signal REG_NEXT : STD_LOGIC_VECTOR(7 downto 0);
    signal CLK_NOT  : STD_LOGIC;

begin
    -- Invert the clock using your NOT gate
    CLK_INV : entity work.not_gate
        port map (
            A => CLK,
            Y => CLK_NOT
        );
    -- Feedback mux: select zero during reset, otherwise D
    -- Assumes mux A is selected for SEL='0'
    -- and mux B for SEL='1'.
    REG_MUX : entity work.mux_8bit
        port map (
            A   => D,                   -- SEL = '0' (normal): pass D
            B   => (others => '0'),     -- SEL = '1' (RESET): zero
            SEL => RESET,
```

```
            Y   => REG_NEXT
        );
    -- Eight storage bits
    REG_BITS : for i in 0 to 7 generate
        BIT_REG : entity work.d_flip_flop
            port map (
                D     => REG_NEXT(i),
                CLK   => CLK,
                RESET => '0',
                Q     => REG_DATA(i)
            );
    end generate;

    Q <= REG_DATA;

end Structural;
```

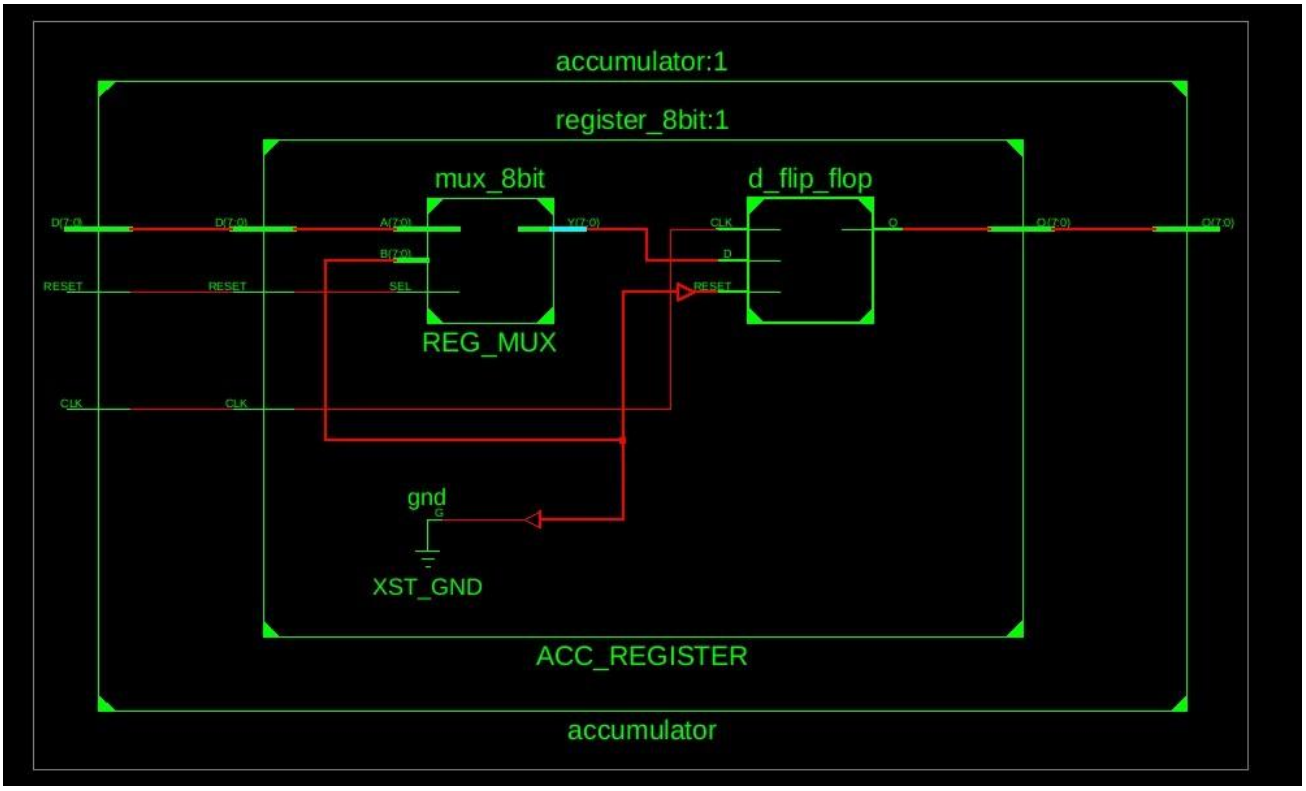


Figure 10: *Accumolator Full Structure*

Source file: zero_flag.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity zero_flag is
    Port (
        A     : in  STD_LOGIC_VECTOR(7 downto 0);
        ZERO  : out STD_LOGIC
    );
end zero_flag;

architecture Structural of zero_flag is
    component not_gate
        Port (
            A : in  STD_LOGIC;
            Y : out STD_LOGIC
```

```
        );
    end component;

    component and_gate
        Port (
            A : in  STD_LOGIC;
            B : in  STD_LOGIC;
            Y : out STD_LOGIC
        );
    end component;
    signal N0, N1, N2, N3 : STD_LOGIC;
    signal N4, N5, N6, N7 : STD_LOGIC;
    signal G1, G2, G3, G4 : STD_LOGIC;
    signal G5, G6 : STD_LOGIC;

begin
    -- Invert every input bit
    INV0: not_gate port map (A => A(0), Y => N0);
    INV1: not_gate port map (A => A(1), Y => N1);
    INV2: not_gate port map (A => A(2), Y => N2);
    INV3: not_gate port map (A => A(3), Y => N3);
    INV4: not_gate port map (A => A(4), Y => N4);
    INV5: not_gate port map (A => A(5), Y => N5);
    INV6: not_gate port map (A => A(6), Y => N6);
    INV7: not_gate port map (A => A(7), Y => N7);
    -- Combine the inverted bits
    AND1: and_gate port map (A => N0, B => N1, Y => G1);
    AND2: and_gate port map (A => N2, B => N3, Y => G2);
    AND3: and_gate port map (A => N4, B => N5, Y => G3);
    AND4: and_gate port map (A => N6, B => N7, Y => G4);

    AND5: and_gate port map (A => G1, B => G2, Y => G5);
    AND6: and_gate port map (A => G3, B => G4, Y => G6);

    AND7: and_gate port map (A => G5, B => G6, Y => ZERO);

end Structural;
```

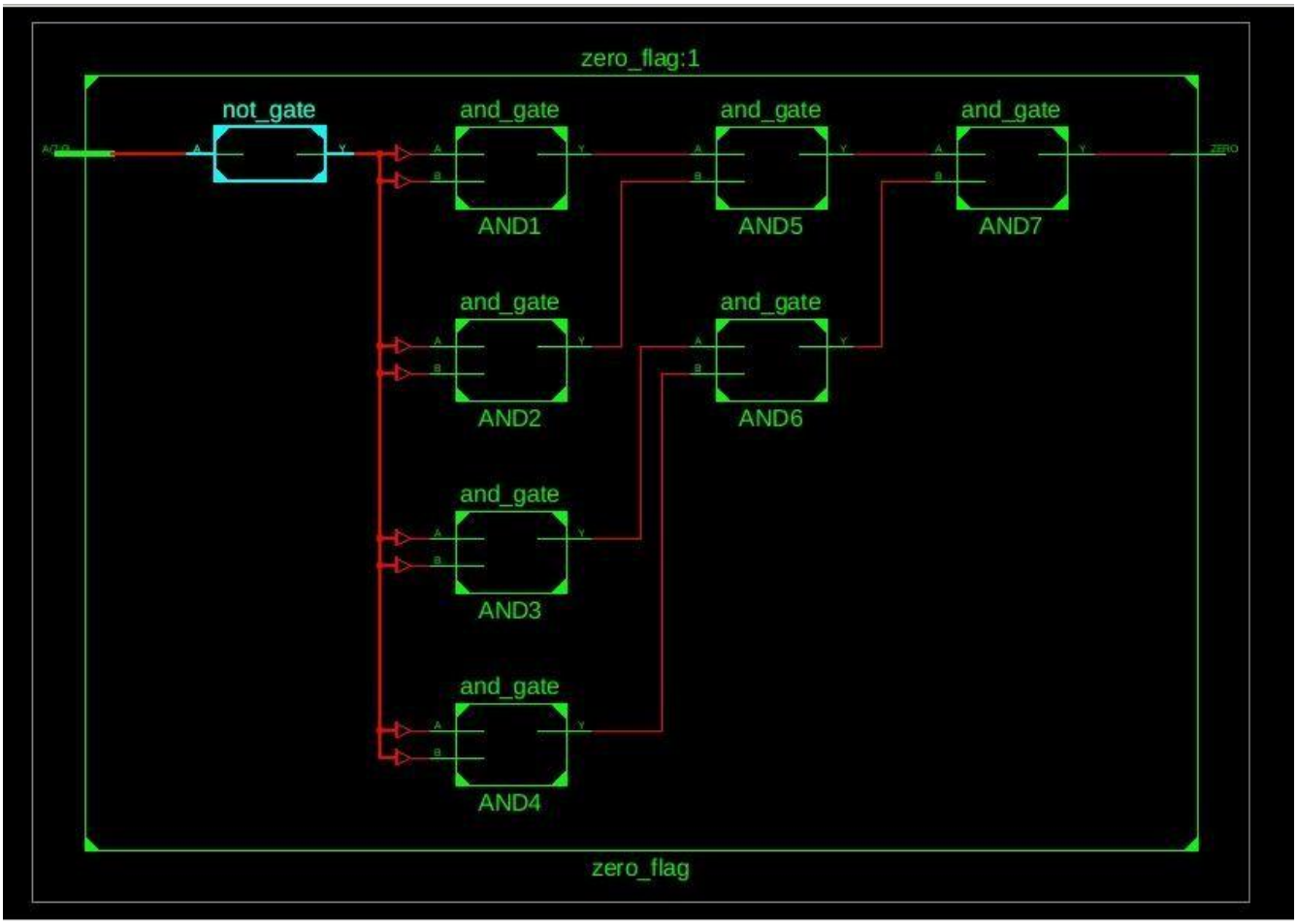


Figure 11: *Zero Flag*

The top-level entity connects the program counter, ROM, instruction decoder, control unit, ALU, accumulator and zero-flag circuit. The output Q is the accumulator value. In the supplied project, Z is generated combinationally from the current accumulator value by zero_flag.vhd.

4. Testbench and Verification

The testbench generates a 20 ns clock. RESET is held high through one rising edge, then deasserted. After each instruction edge, the testbench waits 1 ns before checking the outputs so that signal updates have settled. Assertions compare Q, Z and PC with the expected Set 3 values.

4.1 Expected results for Set 3

Cycle	PC before edge	Instruction	ACC after edge (hex)	ACC (binary)	Z	Next PC
0	—	RESET = 1	0x00	00000000	1	0
1	0	LDI 0x80	0x80	10000000	0	1
2	1	SHR	0x40	01000000	0	2
3	2	SUB 0x10	0x30	00110000	0	3
4	3	ADD 0x0F	0x3F	00111111	0	4
5	4	XOR 0x3F	0x00	00000000	1	5
6	5	ADD 0x07	0x07	00000111	0	6
7	6	SHR	0x03	00000011	0	7

4.2 Testbench code

```
LIBRARY ieee;
```

```
USE ieee.std_logic_1164.ALL;
USE ieee.numeric_std.ALL;

ENTITY cpu_top_tb IS
END cpu_top_tb;

ARCHITECTURE behavior OF cpu_top_tb IS
    COMPONENT cpu_top
        PORT (
            CLK   : IN  std_logic;
            RESET : IN  std_logic;
            Q     : OUT std_logic_vector(7 downto 0);
            Z     : OUT std_logic;
            PC    : OUT std_logic_vector(3 downto 0)
        );
    END COMPONENT;
    -- Inputs
    signal CLK   : std_logic := '0';
    signal RESET : std_logic := '0';

    -- Outputs
    signal Q  : std_logic_vector(7 downto 0);
    signal Z  : std_logic;
    signal PC : std_logic_vector(3 downto 0);
    constant CLK_PERIOD : time := 20 ns;

BEGIN
    -- Instantiate the CPU
    uut : cpu_top
        PORT MAP (
            CLK   => CLK,
            RESET => RESET,
            Q     => Q,
            Z     => Z,
            PC    => PC
        );
    -- Clock generation: 20 ns period
    clk_process : process
    begin
        CLK <= '0';
        wait for CLK_PERIOD / 2;
        CLK <= '1';
        wait for CLK_PERIOD / 2;
    end process;
    -- Set 3 test sequence (LDI, ADD, SHR, SUB, XOR)
    stim_proc : process
    begin
        RESET <= '1';
        wait for CLK_PERIOD; wait for 1 ns;
        assert Q = x"00" and Z = '1' and PC = "0000" report "FAIL: Reset state" severity error;
        RESET <= '0';
        wait for CLK_PERIOD; wait for 1 ns;    -- LDI 0x80
        assert Q = x"80" and Z = '0' and PC = "0001" report "FAIL: LDI 80" severity error;

        wait for CLK_PERIOD;                   -- SHR
        assert Q = x"40" and Z = '0' and PC = "0010" report "FAIL: SHR (addr 1)" severity error;
        wait for CLK_PERIOD;                   -- SUB 0x10
        assert Q = x"30" and Z = '0' and PC = "0011" report "FAIL: SUB 10" severity error;

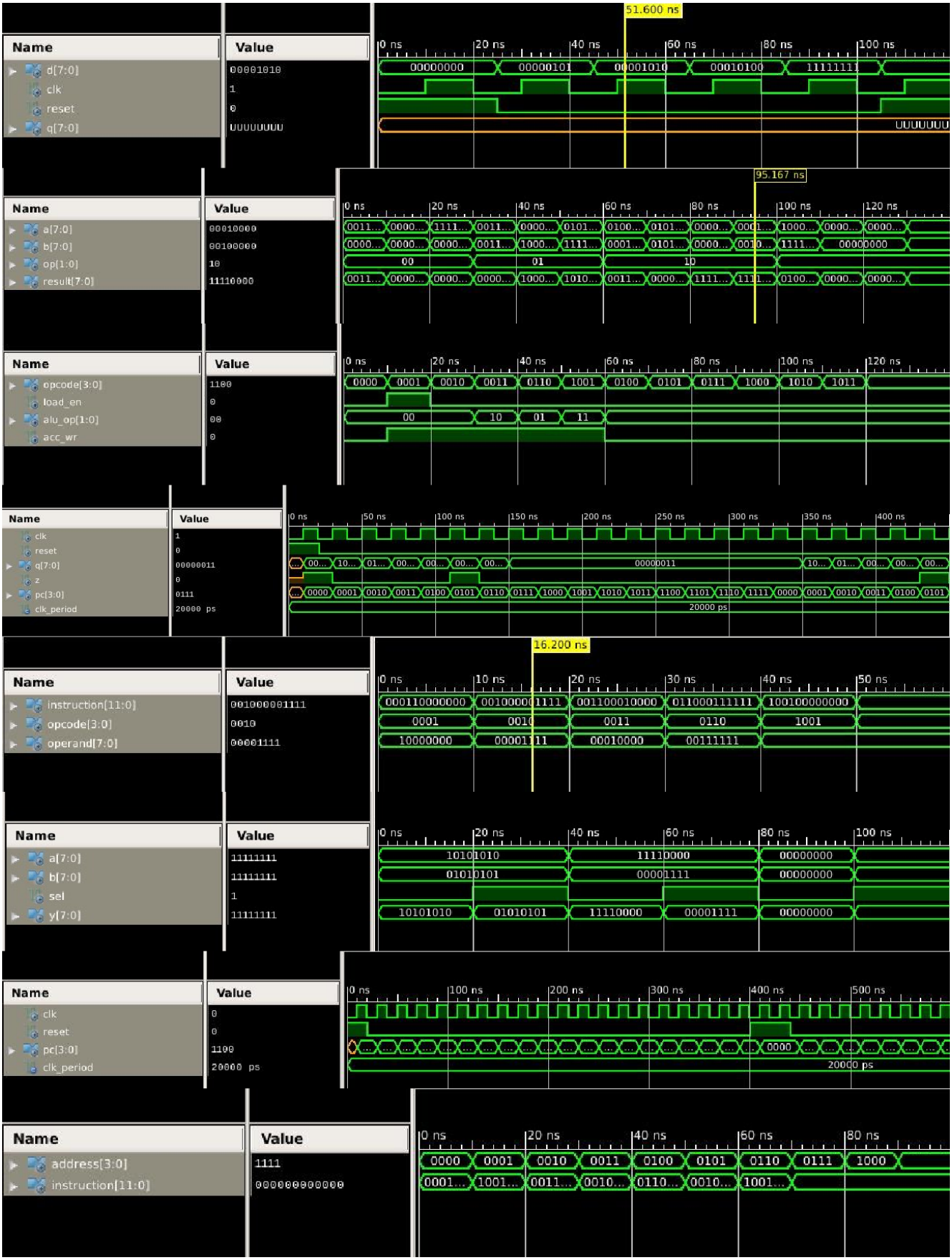
        wait for CLK_PERIOD;                   -- ADD 0x0F
        assert Q = x"3F" and Z = '0' and PC = "0100" report "FAIL: ADD 0F" severity error;
        wait for CLK_PERIOD;                   -- XOR 0x3F
        assert Q = x"00" and Z = '1' and PC = "0101" report "FAIL: XOR 3F" severity error;

        wait for CLK_PERIOD;                   -- ADD 0x07
        assert Q = x"07" and Z = '0' and PC = "0110" report "FAIL: ADD 07" severity error;
        wait for CLK_PERIOD;                   -- SHR
        assert Q = x"03" and Z = '0' and PC = "0111" report "FAIL: SHR (addr 6)" severity error;

        report "Set 3 CPU testbench completed." severity note;
        wait;
    end process;

END behavior;
```

4.3 Simulation screenshots



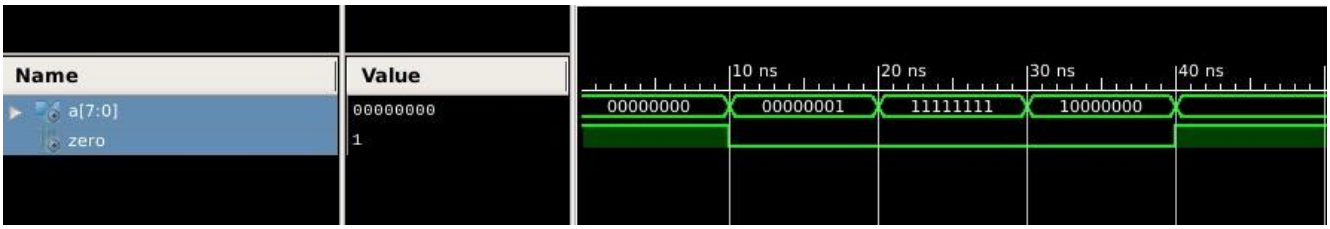


Figure 12: *Simulation*

4.4 Result analysis

The expected result sequence is ACC = 0x80 after LDI, 0x40 after SHR, 0x30 after SUB, 0x3F after ADD, 0x00 after XOR, 0x07 after ADD and 0x03 after the final SHR. The zero flag becomes 1 only when XOR produces 0x00, and it returns to 0 when ADD 0x07 writes a non-zero value. PC advances from 0 to 7, one location per executed instruction.

After running the testbench, compare the waveform and simulator assertions with the expected table above. Do not claim a successful simulation until your own simulator run confirms the outputs.

5. Github Repository

All VHDL source codes, testbenches, and related project files should be uploaded to a GitHub repository.

GitHub Repository Link:

https://github.com/rsf-rawnak/CSE450-VLSI-Labs/tree/main/cpu_design

6. Challenges Faced

The following are reasonable points to discuss if they match the issues encountered during your implementation:

- Instruction encoding: separating the 4-bit opcode from the 8-bit operand and ensuring that each ROM word is exactly 12 bits.
- Sequential versus combinational logic: keeping the ALU combinational while updating PC and ACC only at rising clock edges.
- 8-bit arithmetic: handling subtraction as modulo-256 arithmetic and discarding carry beyond the 8-bit result.
- Zero flag behavior: checking that Z is high when Q is 0x00 and low for non-zero accumulator values.
- Simulation debugging: checking each instruction result against the Set 3 expected-results table.

Edit this section to reflect the actual problems you faced and how you solved them.

7. Conclusion

This project describes a modular single-cycle 8-bit accumulator CPU for Set 3 using VHDL. The CPU supports the mandatory LDI and ADD instructions and the assigned SHR, SUB and XOR instructions. The program ROM stores the exact Set 3 sequence, and the testbench checks the accumulator output, zero flag and program counter after reset and each instruction. Once the simulation has been run, the measured waveform should be compared with the expected results to confirm correct operation.